

Πρώτη Εργασία

Αναγνώρισης Προτύπων Στατιστική Μάθηση

Πριμεράκης Αχιλλεύς

AEM 4688

Κώδικας:

Το Θέμα της εργασίας που επέλεξα ήταν η αναγνώριση χειρόγραφων αριθμών του MNIST dataset την οποία υλοποίησα μέσω των αλγορίθμων k-nn classifier και nearest centroid classifier.

Το πρώτο βήμα ήταν να φορτώσουμε τα αρχεία μέσω της βιβλιοθήκης parquet μιας και τα αρχεία έχουν αυτή τη μορφή, έπειτα μετέτρεψα τις εικόνες που ήταν σε μορφή img σε πίνακες 28 x 28 με στοιχεία τις τιμές από το 0 έως το 255 βάση της πυκνότητας του σχεδίου του αριθμού. Αναλυτικότερα παίρνουμε από την εικόνα τα raw bytes και δημιουργώ ένα αντικείμενο εικόνας η οποία είναι σε greyscale μορφή. Στην συνέχεια μετατρέπω το αντικείμενο σε vector των διαστάσεων που θα χρησιμοποιήσουμε και επιπεδοποιώ τον πίνακα σε διάνυσμα 784 θέσεων.

```
peiramatiko_sunolo = 60000
peiramatiko_sunolo_test = 10000

peiramatiki_lista = []
labels_lista_train = []

for i in range(peiramatiko_sunolo):
    raw_bytes = df.iloc[i]["image"]["bytes"]

    img = Image.open(BytesIO(raw_bytes))
    img = img.convert("L")

    arr = np.array(img)
    vector = arr.reshape(-1)
    peiramatiki_lista.append(vector)

    labels_lista_train.append(df.iloc[i]["label"])

lista_test = []
labels_lista_test = []
for i in range(peiramatiko_sunolo_test):
    raw_bytes = df_test.iloc[i]["image"]["bytes"]

    img = Image.open(BytesIO(raw_bytes))
    img = img.convert("L")

    arr = np.array(img)
    vector = arr.reshape(-1)
    lista_test.append(vector)

    labels_lista_test.append(df_test.iloc[i]["label"])

X_train = np.array(peiramatiki_lista)

X_test = np.array(lista_test)

X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

Το επόμενο τμήμα του Κώδικα είναι η υλοποίηση της ευκλείδειας απόστασης την οποία θα χρησιμοποιήσουμε και στους 2 αλγόριθμους.

```
def euklidia(test, kentro):  
    return np.sqrt(np.sum((test - kentro) ** 2))
```

Επιπλέον θα χρησιμοποιήσουμε και την απόσταση Manhattan για να συγκρίνουμε την ταχύτητα υλοποίησης της.

```
def Manhattan(test, kentro):  
    sum = 0  
    for i in range(len(test)):  
        sum += abs(test[i] - kentro[i])  
    return sum
```

Στο επόμενο βήμα υλοποίησα τον αλγόριθμο των κοντινότερων κέντρων. Αρχικά μέσω του trainset βρίσκουμε τα 10 κέντρα των αριθμών αυτό το κάνουμε βγάζοντας ένα μέσο όρο όλων των διανυσμάτων ενός συγκεκριμένου αριθμού.

```
mesoi_oro_synola = np.zeros((10, 784))  
liasta_gia_dieraiti = np.zeros(10)  
  
for i in range(peiramatiko_sunolo):  
    liasta_gia_dieraiti[labels_lista_train[i]] += 1  
    mesoi_oro_synola[labels_lista_train[i]] += X_train[i]  
  
centroids = np.zeros((10, 784))  
  
for i in range(10):  
    if liasta_gia_dieraiti[i] > 0:  
        centroids[i] = mesoi_oro_synola[i] / liasta_gia_dieraiti[i]
```

Έπειτα ελέγχουμε την αποτελεσματικότητα του αλγόριθμο μέσω του testset αυτό το κάνουμε εντοπίζοντας την μικρότερη απόσταση από κάποιο κέντρο.

```
sum_epitixiwn = 0  
  
for j in range(peiramatiko_sunolo_test):  
    min = float('inf')  
  
    for i in range(10):  
        twrini_euklidia = euklidia(X_test[j], centroids[i])  
        if twrini_euklidia < min:  
            min = twrini_euklidia  
            apotel = i  
  
    if apotel == labels_lista_test[j]:  
        sum_epitixiwn += 1  
  
print(sum_epitixiwn)
```

Η υλοποίηση της K – NN γίνεται με τον παρακάτω κώδικα. Συγκεκριμένα μετατρέπουμε τα δεδομένα μας σε float32 για βελτίωση της ταχύτητας και ορίζουμε τον αριθμό των γειτόνων που επιλέγουμε να προσμετρήσουμε δηλαδή το K μας. Μετά για κάθε αριθμό του testset βρίσκουμε τους κοντινότερους K γείτονες μαζί με τα labels τους. Τέλος μέσω ψηφοφορίας διαλέγουμε τον αριθμό με τις περισσότερες εμφανίσεις στην λίστα των πιο κοντινών.

```
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

sum_epitixiwn = 0

K = 5

for i in range(peiramatiko_sunolo_test):

    kontinoterai_apostaseis = np.full(K, np.inf, dtype=np.float32)
    kontinoterai_labels = np.zeros(K)

    for j in range(peiramatiko_sunolo):

        d = np.sum((X_train[j] - X_test[i]) ** 2)

        if d < kontinoterai_apostaseis.max():
            topikomax = np.argmax(kontinoterai_apostaseis)
            kontinoterai_labels[topikomax] = labels_lista_train[j]
            kontinoterai_apostaseis[topikomax] = d

    if statistics.mode(kontinoterai_labels) == labels_lista_test[i]:
        sum_epitixiwn +=1
```

Αποτελέσματα:

Για να πάρω τα αποτελέσματα χρησιμοποίησα αρχικά ένα μέρος των δεδομένων, 1000 δείγματα για train και 200 δείγματα για test, αυτό το έκανα για να βεβαιωθώ για την σωστή λειτουργία των αλγορίθμων και να συντομεύσω την ταχύτητα υλοποίησής τους. Στην συνέχεια χρησιμοποίησα όλο το dataset το οποίο είχε ως αποτέλεσμα να καθυστερήσει εκθετικά πολύ ο αλγόριθμος.

Nearest centroid classifier:

Τα αποτελέσματα των πλησιέστερων κέντρων έχει ακρίβεια και επιτυχία κοντά στο 82% με υλοποίηση ευκλείδειας απόστασης. Αντίθετα στον υπολογισμό της απόστασης με την συνάρτηση Manhattan έχουμε ακρίβεια της τάξης των 66.8%. Πιο αναλυτικά:

	Επιτυχίες	Χρόνος	Ποσοστό
<i>Euclidean</i>	8203	0.55	82%
<i>Manhattan</i>	6685	22.9	66.8%

Όπως μπορούμε να δούμε και από τον πίνακα καταλαβαίνουμε ότι η Ευκλείδεια συνάρτηση απόστασης είναι πολύ καλύτερη και σε χρόνο αλλά και σε ποσοστό επιτυχιών πράγμα που την κάνει μονόδρομο για την υλοποίηση της K-NN.

k-nn classifier:

Για να ελέγξουμε τις ακρίβειες που είναι ικανό να φτάσει ο K-NN θα χρειαστεί να αλλάξουμε τις τιμές του K. Δηλαδή τον αριθμό των κοντινών γειτόνων του trainset που θα συμπεριλάβουμε για την “ψηφοφορία”. Στον κώδικα που υλοποίησα για τα γείτονες κάτω των 5 υπάρχει σημαντική αύξηση των 10 λεπτών από το 2 έως μέχρι και το 5 ενώ στην συνέχεια σταθεροποιείται. Συγκεκριμένα:

K	Επιτυχίες	Χρόνος	Ποσοστό
2	9642	65 min	96.4%
3	9647	72 min	96.4%
5	9655	82 min	96.5%
6	9671	89 min	96.7%
7	9693	96 min	96.9%

Όπως βλέπετε από τον πίνακα ο K-NN είναι πολύ πιο ακριβείς από τον αλγόριθμο των κοντινότερων κέντρων όμως αυτό έρχεται με το κόστος της κολοσσιαίας αύξησης της πολυπλοκότητας και του χρόνου.

Συμπεράσματα:

Συγκρίνοντας τους δυο αλγόριθμους καταλαβαίνουμε την μεγάλη διαφορά τους στην προσέγγιση επίλυσης του προβλήματος. Τα κοντινότερα κέντρα προσεγγίζει το πρόβλημα βασιζόμενο σε 10 κέντρα ενώ στον K-NN η προσέγγιση γίνεται ελέγχοντας και τα 60.000 δείγματα του trainset. Με άλλα λόγια ο

αλγόριθμος των Centroid θα κάνει $10 * 10.000 = 100.000$ συγκρίσεις, ενώ από την άλλη ο K-NN θα κάνει $10.000 * 60.000 = 600.000.000$ συγκρίσεις.

Χαρακτηριστικά Παραδείγματα:

Nearest centroid classifier:

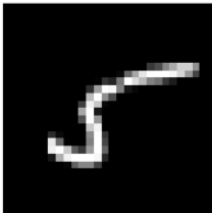
Ευκλείδεια:

Λάθος Προβλέψεις:

Real: 9, Pred: 1



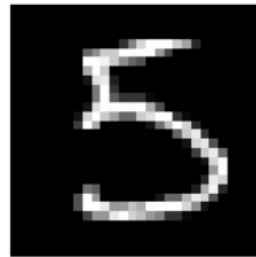
Real: 5, Pred: 1



Real: 2, Pred: 0



Σωστές Προβλέψεις:



Manhattan:

Λάθος Προβλέψεις:

Real: 4, Pred: 1



Σωστές Προβλέψεις:



Real: 6, Pred: 1



Real: 9, Pred: 1



K-NN:

K = 3

Real 4, Pred: 6



K = 5

Real 9, Pred: 8



K = 7

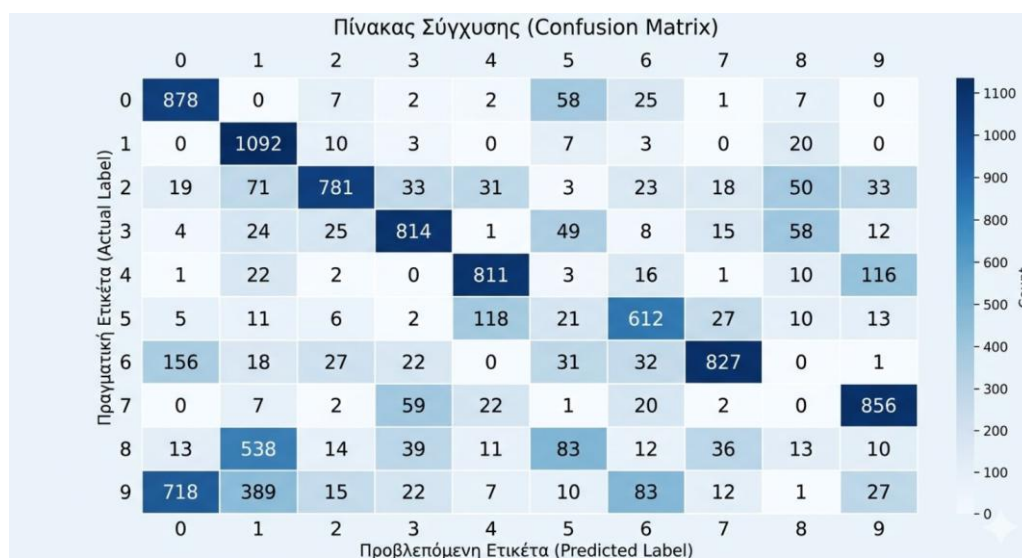
Real 4, Pred: 9



Οι παραπάνω προβλέψεις έγιναν λάθος λόγω του γραφικού χαρακτήρα των δειγμάτων, δεν είναι σπάνιο να συναντάμε παραδείγματα που εμείς θα αναγνωρίζαμε εύκολα με το μάτι αλλά ο αλγόριθμος αποτυγχάνει διότι στην μετατροπή σε ψηφιακή μορφή είναι πολύ δύσκολο ο υπολογιστής να διακρίνει την διαφορά λόγω επίσης του σφάλματος πληροφορίας. Από την άλλη πλευρά υπάρχουν και χαρακτήρες που ακόμα και ανθρώπινο μάτι δεν θα μπορούσε να αναγνωρίσει.

Πίνακες Σύγχυσης:

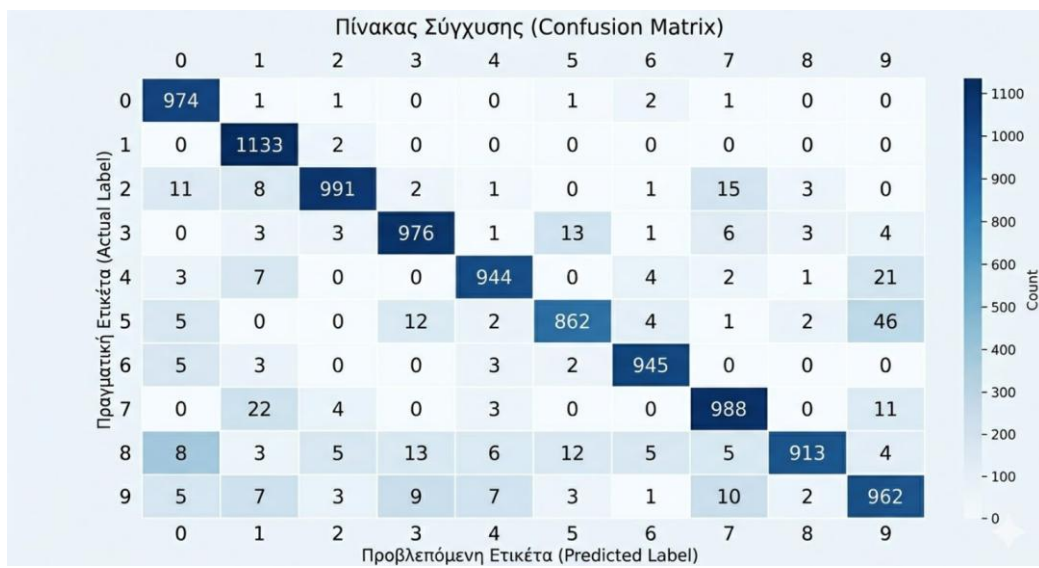
Ευκλείδεια:



Manhattan:



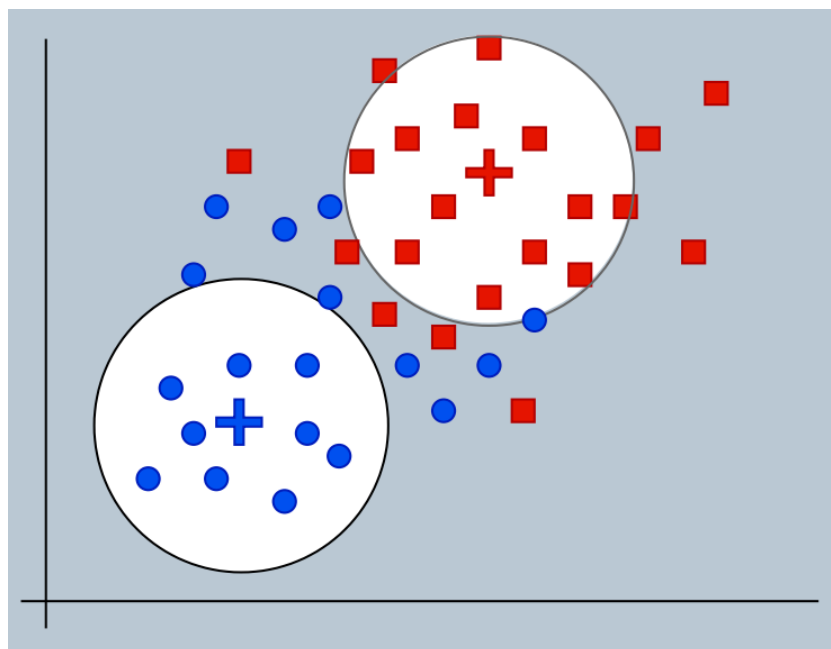
K = 5



Όπως παρατηρούμε στον πίνακα σύγχυσης, τα μεγαλύτερα ποσοστά σφάλματος εντοπίζονται στα ζεύγη (4, 9) και (3, 8). Αυτό οφείλεται στη μεγάλη οπτική ομοιότητα των συγκεκριμένων χειρόγραφων χαρακτήρων, γεγονός που καθιστά την Ευκλείδεια απόσταση μεταξύ τους πολύ μικρή.

Δική μου προσθήκη:

Ύστερα από την μελέτη και υλοποίηση των αλγορίθμων κοντινότερων κέντρων και των κοντινότερων γειτόνων αποφάσισα να δημιουργήσω έναν αλγόριθμο που θα χρησιμοποιήσει και τις 2 προσεγγίσεις. Δηλαδή έφτιαξα έναν υβριδικό αλγόριθμο οποίος υπό κάποιες συνθήκες θα υλοποιήσει τον αλγόριθμο nearest centroids ενώ υπό άλλες θα χρησιμοποιήσει τον K – NN. Συγκεκριμένα για αρχή επεξεργάζομαι την ταξινόμηση με την απόσταση τους από το κοντινότερο κέντρο. Εάν αυτή η απόσταση είναι μικρότερη από ένα συγκεκριμένο όριο, δηλαδή δεν ήμαστε βέβαιοι κατά μεγάλο ποσοστό ότι ανήκει σε μια κατηγορία, τότε το δείγμα εισάγεται στην λεγόμενη “Γκρίζα ζώνη”. Όπως βλέπετε και από το διάγραμμα παρακάτω αυτή η περιοχή στο διάγραμμα είναι για την λεγόμενη κατηγορία αναμφίβολων δειγμάτων, δηλαδή αυτά που είναι δύσκολο να καταταχθούν στην σωστή κατηγορία. Σε αυτή την περίπτωση υλοποιούμε τον αλγόριθμο K – NN ο οποίος όπως έχει αποδειχθεί είναι πολύ πιο ακριβής από αυτόν των κοντινότερων κέντρων αλλά πολύ πιο αργός. Έτσι με αυτόν τον τρόπο μπορούμε να εκμεταλλευτούμε τα πλεονεκτήματα και των 2 αλγορίθμων. Την ακτίνα για το threshold την βρήκα μέσω ενός αλγορίθμου υπολογίζοντας την μέση απόσταση των δειγμάτων από την κανονική τους κατηγορία για το 70%. Τελικά 6.9 ήταν αυτή η απόσταση.



Κώδικας:

Το πρώτο βήμα για την ολοκλήρωση του υβριδικού αλγορίθμου ήταν να βρούμε το όριο (Threshold) βάση το οποίο θα διαχωρίζουμε αν τα δείγματα θα στοιχίσουμε βάση του nearest centroid ή K – NN. Για την εύρεση αυτού του ορίου όπως θα δείτε και στον κώδικα χρειάστηκε αρχικά να υπολογίσουμε όλες τις αποστάσεις των train δειγμάτων από τα κέντρα τους, και να τα προσθέσουμε σε μια λίστα. Έπειτα με την χρήση της συνάρτησης percentile βρίσκω το όριο που χωρίζει το 70% ή την εκάστοτε πλειοψηφία και το υπόλοιπο 30% ή την εναπομένοντα μειοψηφία. Το όριο αποδείχθηκε ότι είναι 6.9.

```
apostaseis_olwn = []

for j in range(len(X_train)):
    apostaseis_olwn.append(euklidia(X_train[j], centroids[labels_lista_train[j]]))

threshold = np.percentile(apostaseis_olwn, 70)
print(threshold)
```

Στην συνέχεια ολοκληρώνουμε τον αλγόριθμο αρχίζοντας βέβαια με τον υπολογισμό της ευκλείας απόστασης από τα centroids και το σημείο, αν αυτή είναι λιγότερο από 6.9 τότε ταξινομείτε το στοιχείο απευθείας στο κέντρο του χωρίς να προβούμε σε άλλο υπολογισμό και να επιβαρύνουμε άσκοπα το πρόγραμμα. Σε άλλη περίπτωση εάν η απόσταση είναι άνω του threshold του 6.9 τότε βρισκόμαστε στην γκρίζα ζώνη και υλοποιούμε τον k-NN με ίδιο τρόπο με πριν. Με αυτό τον τρόπο καταφέρνουμε να διαχειριστούμε τα εύκολα αναγνωρίσιμα δείγματα και την απόλυτη πλειοψηφία με τον ελαφρύ αλγόριθμο nearest centroid και τα δύσκολα αναγνωρίσιμα δείγματα με τον πιο εξειδικευμένο και ακριβή αλλά πολύ βαρύ k-NN.

```

sum_epitixiwn = 0
start_time = time.perf_counter()
pinakas_syn = np.zeros((10,10), dtype=int)

for j in range(peiramatiko_sunolo_test): # 10.000
    min = float('inf')
    apotel = -1

    for i in range(10):
        twrini_euklidia = np.sum((X_test[j] - centroids[i]) ** 2)
        if twrini_euklidia < min:
            min = twrini_euklidia
            apotel = i

    if min > 6.9: #edw einai h aktina gia thn griza zvnh
        k = 3

        kontinoterai_apostaseis = np.full(k, np.inf, dtype=np.float32)
        kontinoterai_labels = np.zeros(k)

        max_dist_in_k = np.inf

        for siggrisi in range(peiramatiko_sunolo):

            d = np.sum((X_train[siggrisi] - X_test[j]) ** 2)

            if d < max_dist_in_k:
                topikomax = np.argmax(kotinoterai_apostaseis)
                kontinoterai_labels[topikomax] = labels_lista_train[siggrisi]
                kontinoterai_apostaseis[topikomax] = d
                max_dist_in_k = kontinoterai_apostaseis.max()

        apotel = statistics.mode(kotinoterai_labels)

    if apotel == labels_lista_test[j]:
        sum_epitixiwn += 1
    pinakas_syn[labels_lista_test[j]][int(apotel)] += 1

    if j % 1000 == 0:
        print(j)
        print(time.ctime())

print(sum_epitixiwn)
end_time = time.perf_counter()
print('Time: ', end_time - start_time)

df_cm = pd.DataFrame(pinakas_syn, index=range(10), columns=range(10))

```

Αποτελέσματα:

	<i>K</i>	<i>Επιτυχίες</i>	<i>Χρόνος</i>	<i>Threshold</i>	<i>Ποσοστό</i>
	3	9702	43 min	8	97%
	5	9688	45 min	8.3	96.8%
	3	9709	37 min	6.9	97.0%
	5	9717	39 min	6.9	97.1%
	7	9693	42 min	6.9	96.9%

Όπως μπορούμε να καταλάβουμε και από τα αποτελέσματα ο χρόνος έχει μειωθεί κατά πολύ το οποίο είναι αξιοσημείωτο αποτέλεσμα. Με άλλα λόγια, ο υβριδικός αλγόριθμος που υλοποιήθηκε κατάφερε να διατηρήσει την κορυφαία ακρίβεια του k-NN, η οποία έφτασε το 97.1% , προσφέροντας ταυτόχρονα μια εξαιρετικά σημαντική μείωση στον χρόνο εκτέλεσης της τάξης του -52.4%.